

UNIT II

Google File System and Map Reduce Framework

Overview

1. GFS: Architecture of GFS, read, write, append algorithms in GFS
2. Fault tolerance in GFS.
3. Map Reduce: Anatomy of a Map Reduce Job Run
4. Architecture, Fault tolerance in Map Reduce, Job Scheduling
5. Shuffle and Sort
6. Task Execution
7. Parallel efficiency of MapReduce
8. Map Reduce Features

Introduction to GFS

What is GFS?

Google File System (GFS) is a scalable, distributed file system developed by Google to handle large data-intensive applications, like web indexing and video processing.

Key Features:

- **Distributed Architecture:** GFS runs on many commodity machines and is designed to be fault-tolerant.
- **Master-Slave Model:**
 - **Master:** Manages metadata (like file names, locations).
 - **Chunkservers:** Store actual data in chunks (usually 64 MB each).
- **Chunk Replication:** Each chunk is replicated (typically 3 copies) for reliability.
- **Write-Once, Read-Many:** Optimized for workloads where files are written once and read many times.
- **Fault Tolerance:** Automatic recovery and replication of lost data or machines.
- **High Throughput:** Designed for fast, large-scale data processing.

Architecture of GFS

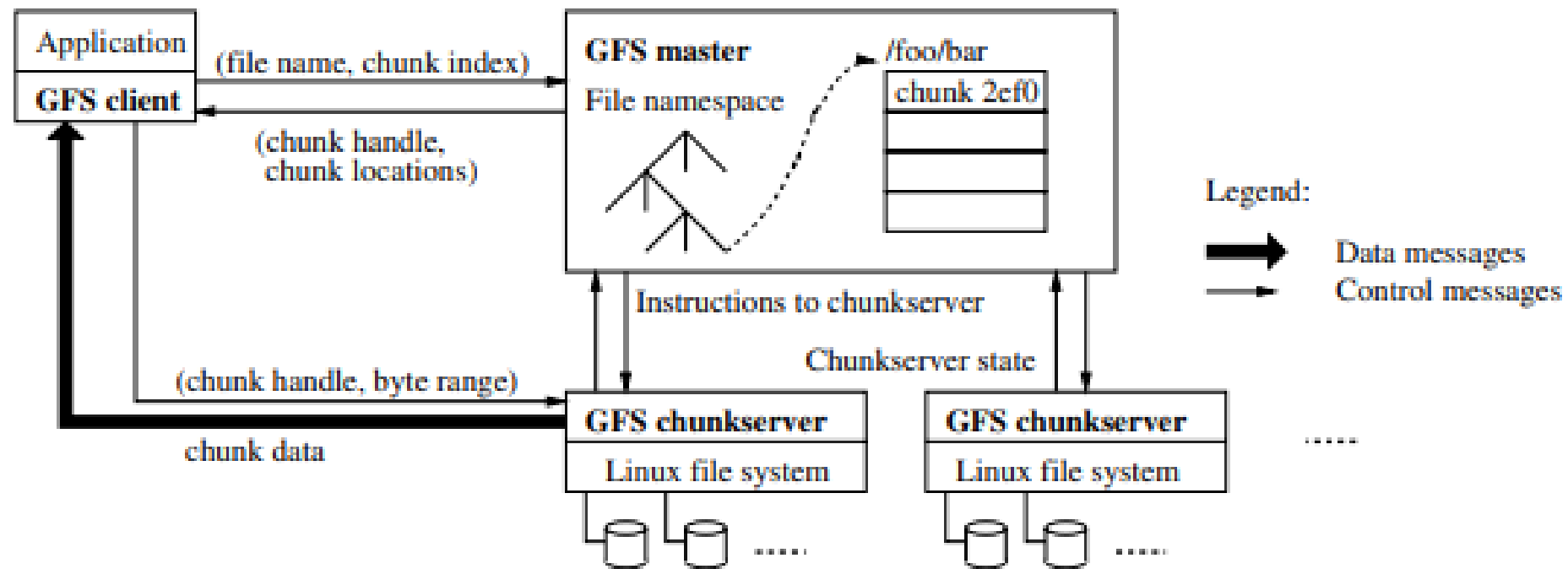


Figure 1: GFS Architecture

Architecture of GFS

Master Server

- The Master server in GFS is the central coordinator responsible for managing the system's metadata, including the namespace (hierarchical file structure), chunk mapping (which chunk belongs to which file), and access control information.
- It maintains in-memory metadata for fast access and uses operation logs and periodic checkpoints for persistence and recovery.
- The master assigns chunk handles, selects chunkservers for new chunks, and monitors the system's health through heartbeat messages from chunkservers.
- It does not store or transfer actual file data, thus reducing its load and making it highly scalable for managing thousands of clients and chunkservers.

Architecture of GFS

Chunk servers

- Chunkservers in GFS are responsible for storing the actual file data in fixed-size chunks of 64 MB, each identified by a unique 64-bit chunk handle.
- These servers manage chunk replicas, usually maintaining at least three copies of each chunk on different nodes to ensure fault tolerance.
- Data is stored as regular Linux files, and chunkservers serve client requests for reads and writes directly.
- They use checksums to detect and repair data corruption and communicate regularly with the master via heartbeats to report their status and chunk versions.
- During write operations, one chunkserver is designated as the primary to coordinate the update sequence among replicas.

Architecture of GFS

Client

- Clients in GFS are user-facing processes or applications that initiate file operations like reading or writing data.
- Instead of constantly interacting with the master, clients only contact the master to obtain metadata, such as the locations and roles of chunk replicas.
- Once they have this information, clients directly communicate with the chunkservers to perform data operations, improving system efficiency and reducing the master's load.
- They cache metadata locally to avoid repeated queries and use a pipelined approach to write data to all chunk replicas, allowing high throughput and minimal master involvement in data flow.

Read, Write and Append Algorithm in GFS

1. Read Algorithm

Goal: Client reads data from a file stored in chunks across chunkservers.

Steps:

- **Client → Master:** Requests chunk location for the specific file and offset.
- **Master → Client:** Returns chunk handle and list of chunkservers holding replicas.
- **Client → Chunkserver:** Chooses the nearest/fastest chunkserver and sends the read request.
- **Chunkserver → Client:** Sends the data directly to the client

Read, Write and Append Algorithm in GFS

2. Write Algorithm

Goal: Client writes data to a file, ensuring consistency across all replicas.

Steps:

- **Client → Master:** Requests chunk information for the file to write.
- **Master → Client:** Returns chunk handle, list of replicas, and designates one as the **primary**.
- **Client → All Chunkservers:** Sends the data (but not the write command) to all replicas via a **pipeline**.
- **Client → Primary Chunkserver:** Sends write command with data offset.
- **Primary → Secondaries:** Primary assigns a serial order and sends the write command to secondaries.
- **All Chunkservers:** Apply the write and send acknowledgment.
- **Client:** After all acknowledgments are received, the write is considered successful.

Read, Write and Append Algorithm in GFS

3. Append Algorithm (Record Append)

Goal: Efficiently append data to a file, often used in log-based systems (e.g., MapReduce).

Steps:

- **Client** → **Master**: Asks for chunk info for the target file.
- **Master** → **Client**: Returns last chunk handle and replica info.
- **Client** → **Chunkservers**: Pushes data to all replicas.
- **Client** → **Primary Chunkserver**: Sends append request.
- **Primary**:
 - Checks if there's enough space in the chunk.
 - If yes, appends data at current end, assigns offset, and tells secondaries to append.
 - If not, tells client to retry with the next chunk.
- **Secondaries**: Perform the same append at the offset given by primary.
- **Client**: Success acknowledged after all replicas finish appending.

Fault tolerance in GFS.

Google File System (GFS) is designed with built-in fault tolerance to handle frequent hardware failures in large-scale distributed environments using inexpensive commodity machines. Here's how GFS achieves fault tolerance:

1. Chunk Replication

- Each chunk is replicated across multiple chunkservers (default: 3 replicas).
- Data remains available even if one or two chunkservers fail.
- The Master ensures replicas are spread across different racks/machines to avoid correlated failures.

2. Master Fault Tolerance

a. Operation Logging & Checkpoints

- Master logs all metadata changes in an operation log.
- Periodically writes a checkpoint to disk.
- On recovery, master reloads from the last checkpoint + log entries after that.

b. Shadow Masters (Read-only)

- Provide read-only access during master unavailability for fault tolerance and high availability.

Fault tolerance in GFS.

3. Chunkserver Heartbeats & Monitoring

Each chunkserver sends regular heartbeat messages to the Master.

The Master:

- Tracks live chunkservers.
- Detects failures quickly.
- Initiates re-replication if a chunkserver goes down.

4. Version Numbers for Consistency

- Each chunk has a version number managed by the Master.
- Helps identify and discard stale or corrupted replicas.
- Prevents inconsistent data from being served or replicated.

5. Re-replication & Recovery

When a chunkserver fails:

- The Master detects the missing chunks.
- It instructs other chunkservers to replicate the chunk to restore the replication factor.
- The system rebuilds lost replicas automatically in the background.

Fault tolerance in GFS.

6. Atomic Record Append Handling

- Ensures that each append operation is performed atomically, so records are written completely or not written at all.
- Prevents partial writes and data corruption in the presence of failures.
- Automatically detects failures and retries the append operation.
- May result in duplicate records, but guarantees that corrupted data is never produced.

7. Data Integrity with Checksums

- Chunkservers maintain checksums for fixed-size data blocks within each chunk.
- On read:
 - During read operations, data is validated by comparing it with the stored checksum.
 - Corrupted data blocks are detected and skipped.
 - Clients obtain correct data by reading from another replica.

Introduction to MapReduce

MapReduce is a programming model and processing framework designed to handle large-scale data processing across many computers in a distributed environment. It was first introduced by Google and later adopted by Hadoop as its core data-processing engine.

The main idea of MapReduce is to break a big computation into two stages:

Map Phase

- Processes input data in parallel.
- Converts raw data into intermediate key–value pairs.

Reduce Phase

- Takes the intermediate data from the map phase.
- Groups values by key and performs aggregation or summarization.

Introduction to MapReduce

MapReduce automatically handles:

- Parallel processing
- Data distribution across nodes
- Fault tolerance (automatic re-execution on failure)
- Data shuffling and sorting
- Load balancing and scheduling

Because of these features, developers only write the `map()` and `reduce()` functions, while the MapReduce framework manages everything else behind the scenes.

It is widely used for tasks like : Word counting, Log analysis, Index building, Large dataset summarization

Example Workflow

- **Input Data:** A large dataset (e.g., text documents).
- **Map Function:** Each document is processed to generate a list of words and their counts (e.g., ("word", 1)).
- **Shuffle and Sort:** The key-value pairs are grouped by key.
- **Reduce Function:** The counts for each word are summed up to produce the final word count

Map Reduce Example

INPUT FILES

This is an
Apple

Apple is red
in color

Anatomy of a Map Reduce job run

A MapReduce job consists of several stages, from job submission to final output, all coordinated by the **MapReduce Master** and executed over data stored in **GFS** or **Hadoop** . Here's a step-by-step breakdown:

1. Job Submission

- ▶ **Client:** The user submits a MapReduce job through a client application.
- ▶ **Job Configuration:** The client specifies parameters such as input and output paths, data formats, and the map and reduce functions.

2. Job Initialization

- ▶ **Job Tracker:** The submission triggers the Job Tracker (or Resource Manager), which manages job execution.
- ▶ **Task Assignment:** The Job Tracker divides the job into map and reduce tasks and assigns them to available Task Trackers (or Node Managers) based on resource availability.

Anatomy of a Map Reduce job run continued...

3. Input Splitting

- ▶ **Data Splits:** The input data is divided into splits, which are the chunks of data that will be processed by individual map tasks.
- ▶ **Input Format:** The specified input format determines how data is read and interpreted for processing.

4. Mapping Phase

- ▶ **Map Tasks:** Each split is processed by a separate map task running on different nodes.
- ▶ **Processing:** The map function transforms input records into intermediate key-value pairs.
- ▶ **Output:** Intermediate results are temporarily stored on local disk for further processing.

5. Shuffling and Sorting

- ▶ **Shuffle Phase:** Intermediate key-value pairs are transferred to reducers so that all values for the same key are grouped together.
- ▶ **Sorting:** Data is sorted by key to enable efficient aggregation during the reduce phase.

Anatomy of a Map Reduce job run...

6. Reducing Phase

- ▶ **Reduce Tasks:** Each unique key from the shuffled data is assigned to a reduce task.
- ▶ **Aggregation:** The reduce function processes grouped values, performing operations such as summation, averaging, or concatenation.
- ▶ **Output Generation:** Final results are generated as key-value pairs.

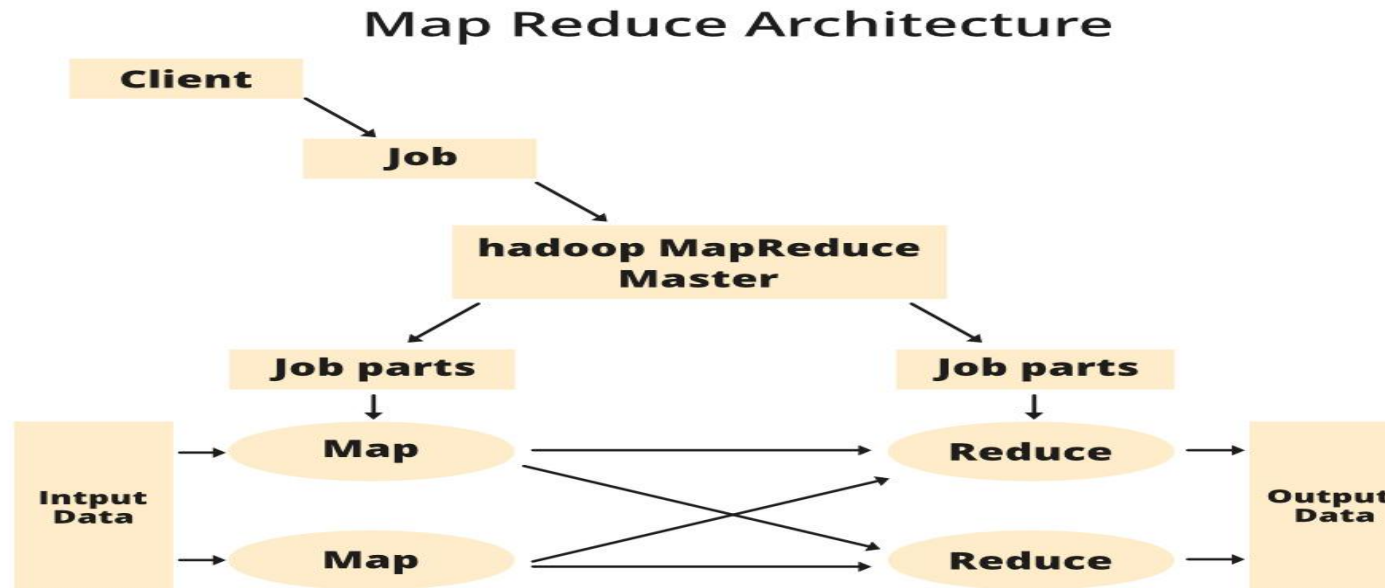
7. Output Writing

- ▶ **Final Output:** Results from the reduce tasks are written to the specified output location in GFS or another distributed file system.
- ▶ **Completion Notification:** Once all tasks are finished, the Job Tracker updates the job status.

8. Cleanup

- ▶ **Temporary Data:** Intermediate data stored locally during mapping and shuffling is removed to free resources.
- ▶ **Job Tracking:** Logs, performance metrics, and any errors are recorded for monitoring and debugging.

Map Reduce Architecture



Core Components of MapReduce Architecture

The MapReduce Architecture follows a master–slave model, where the Job Tracker (master) coordinates tasks and Task Trackers (slaves) execute them on cluster nodes.

Map Reduce Architecture continued...

1. Client

The Client is the entry point into MapReduce. It submits the job for execution by packaging the Mapper and Reducer logic into a JAR file and specifying the input and output paths. After submission, the Client's role ends.

2. Job

- A Job represents the complete processing request from the client.
- Internally, it is divided into job parts (tasks).
- Each job part is assigned to either the Map or Reduce phase.

3. Hadoop MapReduce Master

The Master Node coordinates job execution. It:

- Accepts jobs from the Client.
- Splits them into job parts.
- Assigns these parts to the Map and Reduce tasks.
- Tracks execution progress and reassigns failed tasks.

Map Reduce Architecture continued...

4. Job Parts (Tasks)

- Every job is divided into smaller units called job parts.
- Map job parts: Process input data splits into intermediate key–value pairs.
- Reduce job parts: Aggregate intermediate results into final outputs.

5. Map Phase

- Input data is split and given to Map tasks.
- Each Map task processes its local split and produces intermediate results in the form of key–value pairs.

6. Shuffle & Sort (Between Map and Reduce)

- Intermediate key–value pairs from the Map phase are grouped by key.
- Sorting ensures ordered keys before passing them to the Reducers.

7. Reduce Phase

- Reducers take grouped keys and values from Shuffle & Sort.
- They aggregate them to produce the final output data, which is written back to HDFS.

Fault Tolerance in Map Reduce

MapReduce is designed with robust fault tolerance mechanisms to handle failures gracefully in large-scale distributed environments. Since failures are common in commodity hardware clusters, the framework ensures reliable execution.

Faults in MapReduce

In a distributed environment, faults are inevitable due to the scale of nodes and tasks. Faults in MapReduce can occur at multiple levels:

1 Node/Hardware Faults

- **Node Crash:** Any worker node executing a Map or Reduce task may fail due to hardware issues like CPU failure, overheating, or power outage.
- **Disk Failure:** Storage devices may fail, making the blocks/chunks stored on them unavailable.
- **Network Failures:** Network partitions, high latency, or dropped packets can prevent communication between master and worker nodes, delaying or halting task execution.
- **Impact:** Node failure can interrupt a task in progress, but replication and task rescheduling help mitigate the impact.

Fault Tolerance in Map Reduce continued...

2 Task/Software Faults

- **Task Failure:** Individual Map or Reduce tasks can fail due to runtime errors, invalid input data, exceptions, or resource exhaustion.
- **Software Crashes on Nodes:** The local task management software may crash, temporarily disabling the node from contributing to the job.
- **Impact:** A task failure does not halt the entire job; MapReduce can retry tasks on other nodes.

3 Data-Level Faults

- **Corrupt Blocks/Chunks:** Errors during storage, transmission, or computation can corrupt data.
- **Partial Writes/Incomplete Output:** A node may fail during writing intermediate Map outputs or final Reduce results.
- **Impact:** Reading corrupt or incomplete data can result in incorrect computation; therefore, integrity verification is critical.

4 Master-Level Faults

- **Master Node Failure:** The Job Tracker or Resource Manager manages task scheduling; in GFS, the Master manages metadata. Failure of the master can disrupt scheduling and coordination.
- **Impact:** Master node failure could stop job progress or make metadata temporarily unavailable. Checkpointing and standby masters mitigate this.

Fault Tolerance in Map Reduce continued...

Fault Tolerance Mechanisms

1 Data Replication

- **How it works:** Input and intermediate data blocks/chunks are stored on multiple nodes (default replication factor = 3).
- **Why it matters:** If one replica becomes unavailable due to node or disk failure, MapReduce reads from other replicas.

2 Task Re-Execution

- **How it works:** If a Map or Reduce task fails, the system automatically reschedules it on another node.
- **Why it matters:** Software errors or hardware failures affecting a task do not stop the job.

3 Speculative Execution

- **How it works:** Tasks that are running slower than expected (stragglers) are duplicated on other nodes.
- **Why it matters:** Prevents overall job slowdown due to a few slow nodes.

Fault Tolerance in Map Reduce continued...

4 Checkpointing and Logging

- **How it works:** Masters maintain logs of task execution, while distributed file systems maintain write-ahead logs for metadata.
- **Why it matters:** Enables recovery of partially completed tasks and prevents loss of metadata in case of crashes.

5 Data Integrity Verification

- **How it works:** Checksums are computed for each data block/chunk. During read operations, data is verified against its checksum.
- **Why it matters:** Detects and avoids using corrupted data; if a block fails verification, another replica is used.

6 Master Node Fault Handling

- **How it works:** Metadata and task progress logs are periodically checkpointed. Some systems use standby master nodes that can take over if the primary master fails.
- **Why it matters:** Ensures the cluster can continue operations with minimal downtime.

Job Scheduling in MapReduce

Job scheduling in MapReduce is a crucial component that determines how tasks are allocated and executed across a distributed cluster. Here's a detailed overview of the job scheduling process:

1. Job Submission

- **Client Submission:** A client submits a MapReduce job to the Job Tracker (or Resource Manager), specifying the job's configuration, including input and output paths and the map and reduce functions.

2. Job Initialization

- **Job Tracker Role:** The Job Tracker initializes the job by assigning it a unique identifier and setting up the job configuration.
- **Task Creation:** The job is divided into smaller tasks, specifically map tasks and reduce tasks, based on the input splits and the defined functions.

3. Task Scheduling

- **Task Queue:** The Job Tracker maintains a queue of tasks, prioritizing them based on various factors, such as resource availability and task dependencies.
- **Resource Availability:** The Job Tracker checks the status of Task Trackers (or Node Managers) to determine which nodes have available resources to execute tasks.

Job Scheduling in MapReduce

4. Task Assignment

- **Locality Optimization:** The scheduler aims to assign tasks to nodes where the data resides (data locality) to reduce network I/O and improve performance.
- **Load Balancing:** The Job Tracker balances the load across the cluster by distributing tasks evenly among available nodes.

5. Task Execution

- **Task Execution:** Once assigned, Task Trackers execute the map or reduce tasks. They process the data and generate intermediate results.
- **Progress Reporting:** Task Trackers send periodic heartbeat signals and progress updates to the Job Tracker, allowing it to monitor task execution.

6. Handling Failures

- If a task fails, the Job Tracker detects the failure and reschedules the task on another available node.

Job Scheduling in MapReduce

7. Completion and Output

- **Task Completion:** Once all map tasks are completed, the Job Tracker triggers the reduce tasks.
- **Final Output:** After all reduce tasks finish, the final results are written to the specified output location.

8. Job Monitoring and Logging

- The Job Tracker maintains the status of the job and logs performance metrics, successes, and failures for monitoring.

Shuffle and Sort Phase in MapReduce

The Shuffle and Sort phase is an intermediate step between the mapping and reducing stages in MapReduce. Its overall purpose is to **redistribute, group, and organize the intermediate key–value pairs** generated by the map tasks so that all values associated with the same key are brought together and presented in sorted order for efficient reduce-side processing.

1. Shuffle Phase

- **Data Transfer:** After the map tasks complete, their intermediate key–value pairs are transferred over the network from the mapper nodes to the nodes where the reduce tasks will execute.
- **Partitioning:** A partitioning function determines which reducer should receive each key. This ensures that all values corresponding to a specific key are directed to the same reduce task.
- **Network Efficiency:** The shuffle mechanism minimizes network traffic by grouping, merging, and compressing intermediate data during transfer, improving overall performance in large distributed environments.

Shuffle and Sort Phase in MapReduce

2. Sort Phase

- **Sorting by Key:** Once the shuffled data reaches the reducer nodes, it is sorted based on the key. Small datasets may be sorted in memory, while larger datasets require disk-based external sorting.
- **Grouped Data:** The sorting process ensures that all identical keys appear consecutively, allowing the reducer to process each key sequentially along with its associated list of values.

3. Output of the Shuffle–Sort Phase

- **Input to Reducers:** The result of this phase is a structured set of keys paired with their grouped intermediate values, ready for processing by the reduce function.
- **Example (Word Count):**
Intermediate map output:
("apple", 1), ("banana", 1), ("apple", 1)
After shuffle and sort:
("apple", [1, 1])
("banana", [1])

Parallel efficiency of MapReduce

Parallel efficiency is a measure of how effectively a parallel computing system utilizes multiple processors to perform a task. It is defined as:

$$\text{Parallel Efficiency (E)} = \frac{\text{Speedup (S)}}{\text{Number of Processors (P)}}$$

Where:

Speedup (S) = Time taken to execute the task sequentially / Time taken to execute in parallel.

P = Number of parallel processors (or nodes in MapReduce cluster).

So, efficiency tells us **how well the workload is distributed** among the nodes. Ideal efficiency is **1 (or 100%)**, meaning all processors are fully utilized without idle time.

Parallel efficiency of MapReduce

Key Factors Affecting Parallel Efficiency in MapReduce:

1. Task Granularity:

- Each MapReduce job is split into map tasks and reduce tasks.
- If the tasks are too small, overhead dominates computation, reducing efficiency.
- If tasks are too large, some nodes may finish earlier and sit idle. i.e. affecting parallelism.

2. Data Distribution:

- MapReduce works best when input data is evenly distributed across nodes.
- Uneven distribution (data skew) leads to some nodes processing more data than others which reduces efficiency.

3. Communication Overhead:

- Shuffle phase (transfer of intermediate key-value pairs from mappers to reducers) can create network congestion.
- High network latency decreases efficiency because nodes spend time waiting for data.

4. Stragglers (Slow Tasks):

- A single slow task can delay the completion of the job.
- MapReduce uses speculative execution to reduce this effect.

Parallel efficiency of MapReduce

Ways to Improve Parallel Efficiency

a) Optimize Task Size

- Divide input data into tasks that are not too small (avoiding overhead) and not too large (avoiding idle nodes).
- This ensures that all nodes stay active until the job finishes.

b) Balanced Data Partitioning

- Ensure data is evenly split across nodes to avoid some nodes finishing early while others are still processing.
- Techniques include custom partitioners or preprocessing data to balance sizes.

c) Reduce Communication Costs

- Use combiners to aggregate intermediate results before the shuffle phase.

- Compress data to reduce network transfer.

- Data Locality: Processing data close to where it is stored reduces network overhead and increases efficiency.

d) Speculative Execution

- Detect slow tasks (stragglers) and run duplicates on other nodes.
- This reduces the impact of slow tasks on overall job completion time.

e) Proper Resource Management

- Allocate sufficient CPU, memory, and network bandwidth to nodes.
- Prevents tasks from being delayed due to resource contention, keeping all nodes efficiently busy.

Key Features of MapReduce

1. Scalability

- MapReduce can efficiently process terabytes or petabytes of data by distributing work across hundreds or thousands of nodes.
- Adding more nodes to the cluster allows the system to handle larger datasets without rewriting the program.
- Example: A word count program can run on 10 nodes or 1,000 nodes with minimal code changes.

2. Parallel Processing

- Tasks are divided into independent map and reduce tasks that can execute simultaneously on different nodes.
- This allows the system to significantly reduce total processing time.
- Parallelism is automatic; the programmer does not need to manage threads or synchronization explicitly.

3. Fault Tolerance

- MapReduce is designed to handle hardware failures gracefully.
- If a node fails during execution, the system automatically re-executes the failed tasks on other nodes.
- Input and intermediate data are stored in a distributed file system (like HDFS, GFS), making recovery easier.

4. Data Locality

- MapReduce tries to process data on the node where it is stored, minimizing network traffic.
- Instead of moving large datasets across the network, tasks are sent to nodes containing the data.
- This improves performance and reduces communication overhead.

Key Features of MapReduce

5. Simplified Programming Model

- Programmers only need to implement two functions:
 - **Map()** – processes input data and generates key-value pairs.
 - **Reduce()** – aggregates or summarizes intermediate key-value pairs into final output.
- The system handles task scheduling, data distribution, parallel execution, and fault tolerance, hiding the complexity of distributed computing.

6. Flexibility

- Supports various data types and structures.

7. Automatic Load Balancing

- The framework dynamically assigns tasks to available nodes.
- If a node finishes its tasks early, it can pick up remaining tasks from slower nodes (stragglers), keeping all nodes effectively utilized.

Thank You !